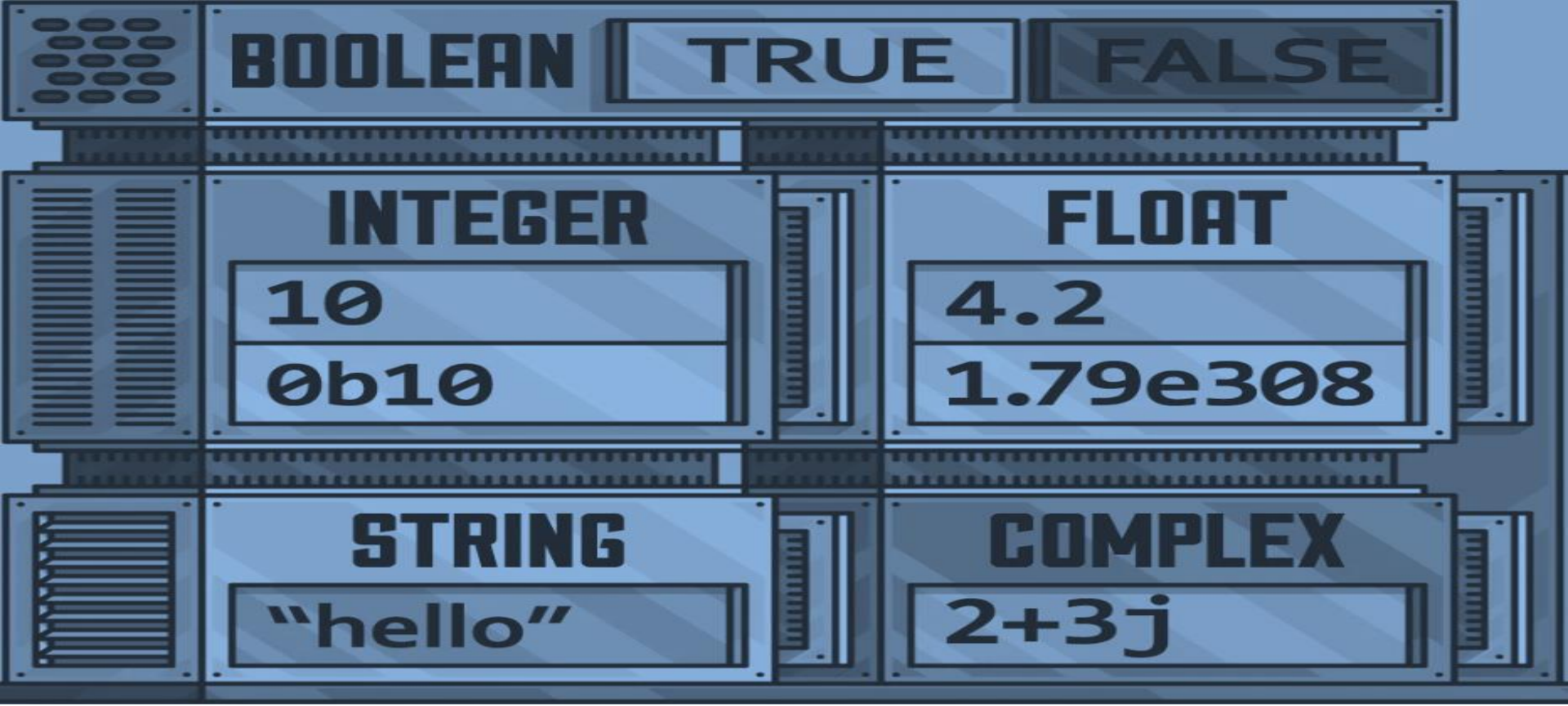


Java Programming

CPT111 – Lecture 2

Erick Purwanto

Xi'an Jiaotong-Liverpool University
西交利物浦大学

CPT111 Java Programming

Lecture 2

Data Types, Math, Client Input, Debugging

Welcome to CPT111 – Lecture 2 !

- Find a friend to discuss today's lecture
- Please join: Lecture 2 In-Class Quiz
- What do you think about the Manga **Chainsaw Man**?
  - I feel so bad for Denji
  - I cannot stand the violence and intense situations
  - I really enjoy the dark humor
  - I haven't read it
  - I love Makima!

Learning Outcomes

- Welcome to **Lecture 2** !
- **Lecture Outcomes:**

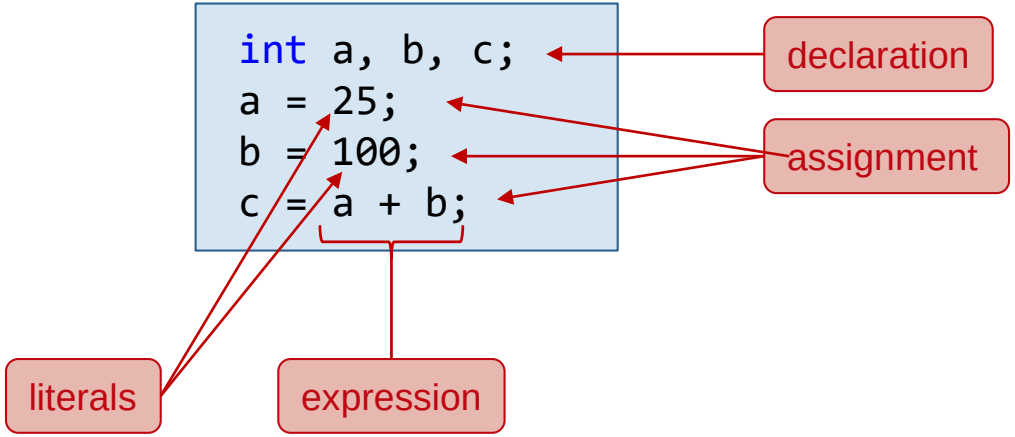
  In this lecture we are going to learn

  - Data types
    - Integer, Floating Point, Boolean, Char, String
    - Arithmetic, Logical, Comparison Operators
  - Math Library, Random Numbers, Casting
  - Program Input and Interactive Programs by converting between types
  - Using Debugger and Visualizer

- Optional **Reading**: Liang Chapter 2

Recap Last Week

- Last week, you have learned some terms in Java
  - Three **variables** are *declared* using the *identifiers* a, b, c with *type* `int`



In-Class Quiz 1: Recap on Variables and Assignments

- Suppose a and b are two different integer values.
  What will happen at the end of these statements?

```
int c = a;
b = c;
a = b;
```

  - a, b, and c will have different values
  - a, b, and c will have the same values: the original value of a
  - a, b, and c will have the same values: the original value of b
  - a, b, and c will have the same values: the original value of c

Data Types

- A **data type** is a set of values, along with operations defined on them
- Five most frequently used data types:

Data Type	Values	Sample Operators	Sample Values
`int`	integers / natural numbers	`+ - * / %`	`25 -1 2147483647`
`double`	floating-point numbers	`+ - * /`	`2.5 -1.0 1.7e308`
`boolean`	boolean values	`&& || !`	`true false`
`char`	characters		`'A' '\n' '&' '2'`
`String`	sequences of characters	`+`	`"Hello" "a \n" "2.5"`

Integers

- An **integer** is a whole number between -2^{31} (-2,147,483,648) and $2^{31} - 1$ (2,147,483,647)
- **Operations** with arithmetic operators:

Expression	Value	Meaning
`-100`	`-100`	negative sign
`2 + 5`	`7`	addition
`2 - 5`	`-3`	subtraction
`5 * 2`	`10`	multiplication
`5 / 2`	?	
`5 % 2`	?	
`2 + 5 * 2`	?	

In-Class Quiz 2: Integer Division

- What is the output of the following expression: `5 / 2`

  - compile error
  - runtime error
  - 2
  - 3
  - 2.5

编译错误

运行错误

In-Class Quiz 3: Integer Division by 0

- What is the output of the following expression: `1 / 0`
  - compile error
  - runtime error
  - `0`
  - `Infinity`
  - `NaN`

In-Class Quiz 4: Modulo / Remainder

- What is the output of the following expression: `5 % 2`

  - `0`
  - ✓ `1`
  - `2`
  - `10`
  - `0.1`

reminder

In-Class Quiz 5: Precendence

- What is the output of the following expression: `2 + 5 * 2`
  - `7`
  - `9`
  - `10`
  - `12`
  - `14`

More Integer Operators and Long Data Type

- More operations, for example with an expression ex (e.g., ex has value 5):

Expression	Value	Meaning
`ex++`	6	postfix increment (add by 1)
`++ex`	6	prefix increment
`ex--`	4	postfix decrement (minus 1)
`--ex`	4	prefix decrement
`ex += 2`	12	`ex = ex + 2`
`ex *= 10`	50	`ex = ex * 10`

for program clarity, simply choose postfix for shortcut

similarly for other arithmetic operators...

- If `int` is not large enough, you can use **`long`** data type

```
long l = 9223372036854775807;
```

Decimal Numbers

- The **`double`** type represents decimal / floating-point numbers
  - `3.14159` for six-digit approx. of PI, or `6.022E23` for Avogadro's constant 6.022×10^{23}
- **Operations** with arithmetic operators:

Expression	Value	Meaning
`3.14 + 2.0`	`5.14`	addition
`3.14 - 2.0`	`1.14`	subtraction
`3.14 * 2.0`	`6.28`	multiplication
`3.14 / 2`	`1.57`	floating-point division
`5 / 3.0`	`1.6666666666666667`	floating-point division
`7.5 % 3.0`	`1.5`	modulo / remainder
`Math.sqrt(4.0)`	`2.0`	square root

In-Class Quiz 6: Floating-point Division by 0

- What is the output of the following expression: `1.0 / 0.0`
  - compile error
  - runtime error
  - `0`
  - `Infinity`
  - `NaN`

In-Class Quiz 7: Square Root of Negative Number

- What is the output of the following expression: `Math.sqrt(-1.0)`
  - compile error
  - runtime error
  - `0`
  - `Infinity`
  - `NaN`

Boolean

- **Boolean** type only has two values: `true` or `false`
  - from mathematician George Boole

```
public class BooleanValues {
    public static void main(String[] args) {
        boolean a = true;
        boolean b = false;
        System.out.println(a);
        System.out.println(b);
    }
}
```

Boolean/Logical Operators

- Operators defined for boolean:
  - **and**: a && b is true if both a and b are true, and false otherwise
  - **or**: a || b is true if either a or b is true (or both are true), and false otherwise
  - **not**: !a is true if a is false, and false otherwise

a	!a
true	
false	

a	b	a && b	a || b
true	true		
true	false		
false	true		
false	false		

Comparison Operators

- **Comparison** operators that take operands of one type (e.g., `int` or `double`) and produce a result of type `boolean`

operator	meaning	`true`	`false`
==	equal		
!=	not equal		
<	less than		
<=	less than or equal		
>	greater than		
>=	greater than or equal		

  - ***do not*** confuse equality operator == with assignment operator =

In-Class Quiz 8: Boolean and Comparison Operators

- Given an integer variable `month`, give an expression of a legal month:
  - ❑ `(month >  1) && (month <  12)`
  - ❑ `(month >  1) || (month <  12)`
  - ❑ `(month >= 1) && (month <= 12)`
  - ❑ `(month >= 1) || (month <= 12)`
  - ❑ `(month >  0) && (month <  13)`
  - ❑ `(month >  0) || (month <  13)`

Precedence

- Operator **precedence** / order of execution:

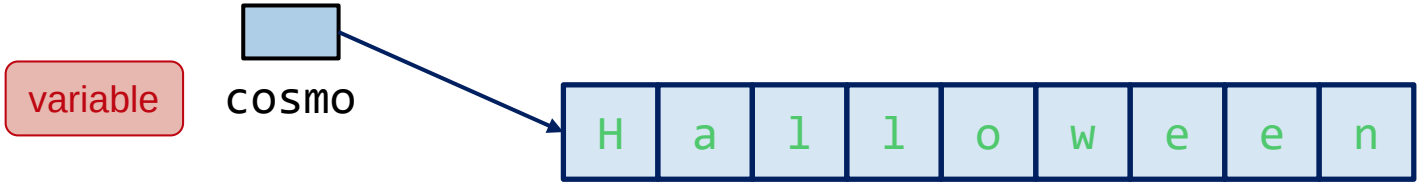
Operators	Precedence
postfix	ex++ ex--
unary	! ++ex --ex
multiplicative	* / %
additive	+ -
relational	< > <= >=
equality	== !=
logical and	&&
logical or	||
assignment	= += -= *= /= %=

for program clarity, simply use parentheses!
(ex **)**
even if it is redundant
e.g. a = (a * b) + 3

Java String

- A **`String`** is a sequence of characters (chars), like the word `"Halloween"`, or a sentence `"A corpse is talking."`
- Create a Java string by writing its chars between double quotes `" "`

```
String cosmo = "Halloween";
```



Concatenation

- Plus **+** operator between strings concatenate them together to make a new, bigger string
  - chars of the first string put together with chars of the second string
  - works with strings stored in variables too

```
String cosmo = "hallo" + "ween";

String bang = "bang";
String exclamation = "!!";
String makima = bang + exclamation;
```

  - we can convert any type of data to a `String` by using the + operator

```
String bangbang = bang + 2;
```

Convert Numbers to Strings

- One simple way to convert is to use the concatenation operator +

```
double d = 2.5;
String s = "" + d;
System.out.println(s);
```

the empty string

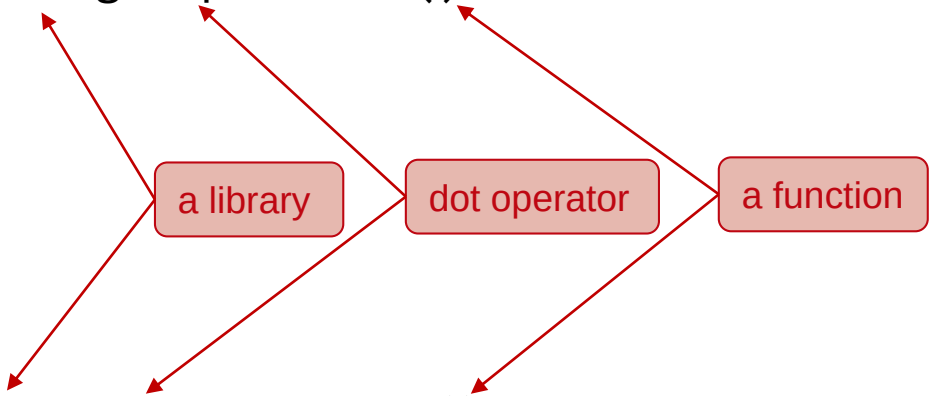
- What is the output?

```
System.out.println(1 + "2");
System.out.println(1 + 2 + "3");
System.out.println("ab" + 12);
System.out.println("ab" + 1 + 2);
System.out.println("ab" + (1 + 2));
```

Convert Strings to Numbers

- Convert a String to an integer with `Integer.parseInt()`

```
String s = "5";
int i = Integer.parseInt(s);
i++;
System.out.println(i);
```



- Convert a String to a double with `Double.parseDouble()`

```
String t = "2.5";
double d = Double.parseDouble(t);
d = d * 3.0;
System.out.println(d);
```

what happens if the string is ***not*** a number?

Math Library

Math Function	Meaning
`double abs(double a)`	absolute value of a
`double max(double a, double b)`	maximum of a and b
`double min(double a, double b)`	minimum of a and b
`double sin(double theta)`	sine of theta
`double cos(double theta)`	cosine of theta
`double tan(double theta)`	tangent of theta
`double toRadians(double degrees)`	convert angle from degrees to radians
`double toDegrees(double radians)`	convert angle from radians to degrees
`double exp(double a)`	exponential (e^a)
`double log(double a)`	natural log ($\log_e$ a, or ln a)
`double pow(double a, double b)`	raise a to the b-th power (a^b)
`long round(double a)`	round a to the nearest integer
`double random()`	random number in [0, 1)
`double sqrt(double a)`	square root of a

Rounding and Casting

- What if we want to round a double to an integer?

```
int i = Math.round(3.14159);
```

  - a compile error!

- We need to convert the long output to an integer first by ***casting***

```
int i = (int) Math.round(3.14159);
```

  - casting a double does ***not*** round it, instead, the fractional part is ignored

Problem Solving: What day is it?

- Open Learning Mall – CPT111 – Week 2: **Exercise #2.1**

Write a Java program that takes a date as input and prints the day of the week that date falls on.

Your program should take three integers: y (year), m (month), and d (day).
For m use 1 for January, 2 for February, and so on.
For output print 0 for Sunday, 1 for Monday, 2 for Tuesday, and so on.

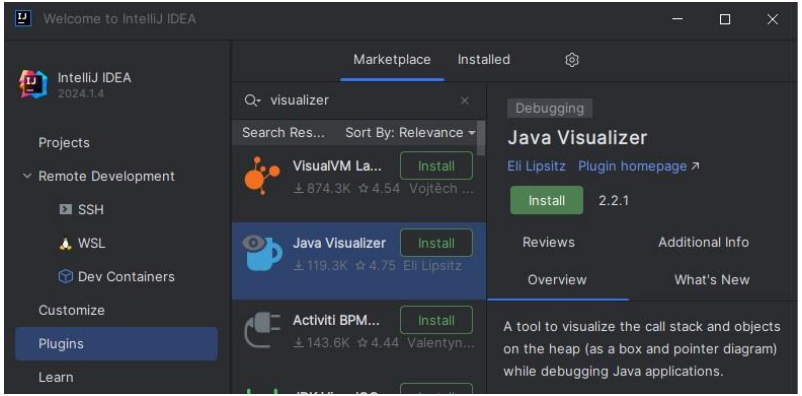
Use the following formulas:

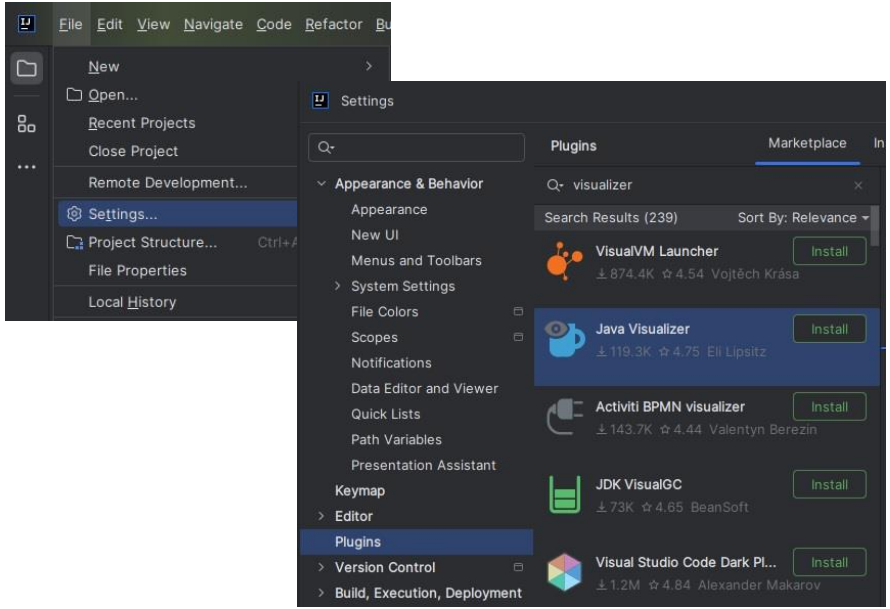
$$
\begin{aligned}
a &= y - (14 - m)/12 \\
b &= a + a/4 - a/100 + a/400 \\
c &= m + 12 \times ((14 - m)/12) - 2 \\
day &= (d + b + (31 \times c)/12) \mod 7
\end{aligned}
$$

  - we will also learn to use **Debugger** and **Visualizer** to help us with debugging

Java Visualizer

- Open IntelliJ IDEA
  - in Project Welcome Screen or Main Menu - Settings
  - search "visualizer" and install **Java Visualizer**





Testing

- You are given one or more test cases
  - there are given test cases and hidden test cases
  - your code is **correct** *if and only if* it passes **all** test cases

Your program should take three integers: y (year), m (month), and d (date).
For m use 1 for January, 2 for February, and so on.
For output print 0 for Sunday, 1 for Monday, 2 for Tuesday, and so on.

Test case 1:
Input:
2018
12
24
Output:
It's day 1 !

```
int y = 2018;
int m = 12;
int d = 24;
```

- Add **more** test cases!
  - create and compute ***your own*** test cases

Buggy Code

- You are given the following formula:

$$
\begin{aligned}
a &= y - (14 - m)/12 \\
b &= a + a/4 - a/100 + a/400 \\
c &= m + 12 \times ((14 - m)/12) - 2 \\
day &= (d + b + (31 \times c)/12) \mod 7
\end{aligned}
$$

- Suppose you wrote the following (buggy) code:

```
int a = y - 14 - m / 12;
int b = a + a/4 – a/100 + a/400;
int c = m + 12 * (14 - m / 12) - 2;
int day = (d + b + 31 * c / 12) % 7;

System.out.println("It's day " + day + " !");
```

  - and the output is wrong:

```
It's day 0 !
```

how can we find the bugs?

Debugging Attempt

- How can **find** and **test** the lines that may contain bugs?
- One way is to `println` the intermediate results, and compare with the correct values computed manually

```
int a = y - 14 - m / 12;
int b = a + a/4 – a/100 + a/400;
int c = m + 12 * (14 - m / 12) - 2;
int day = (d + b + 31 * c / 12) % 7;

System.out.println("It's day " + day + " !");
```

add println(a) here

add println(b) here

add println(c) here

need to modify the code, may forget to remove later... is there a better way?

Using Debugger

- Set a **breakpoint** by clicking the area close to the line numbers
  - pick the line where you want the program execution to stop

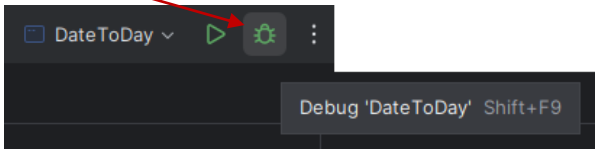
left click

```
3   public static void main(String[] args) {
4
5       int y = 2018;
6       int m = 12;
7       int d = 24;
8
9       int a = y - 14 - m / 12;
10      int b = a + a/4 - a/100 + a/400;
11      int c = m + 12 * (14 - m / 12) - 2;
12      int day = (d + b + 31 * c /12) % 7;
13
14      System.out.println("It's day " + day + " !");
15  }
```

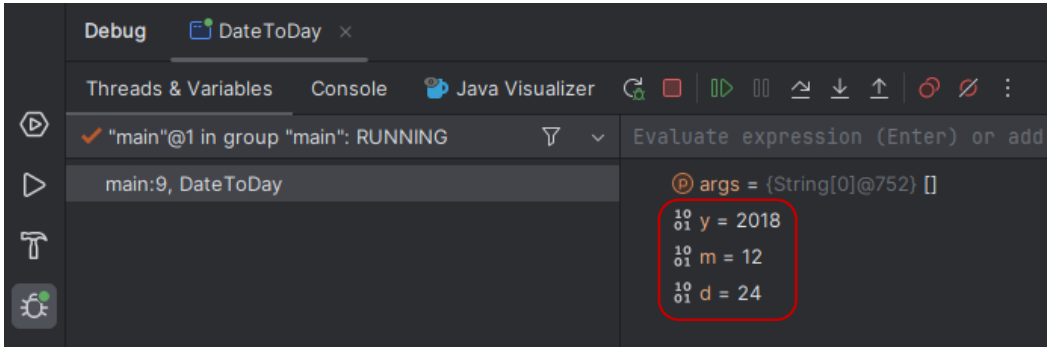
  - after that, the variables will be observable line-by-line

Debugger View

- Click the **Green Bug** icon

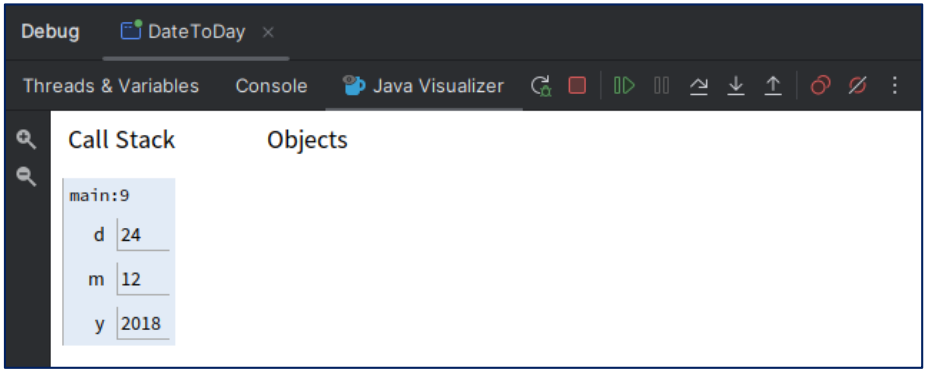


- You can see the current state of the program (the variables y, m, d) ***before*** line 9 is executed (before variable a is initialized with the evaluation of the expression from the formula)



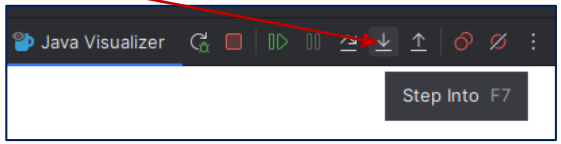
Java Visualizer

- Alternatively, we can use **Java Visualizer** to illustrate the state of the program
  - currently, there are only boxes to show all the variables, but the illustration will become more helpful later as we code more complicated program

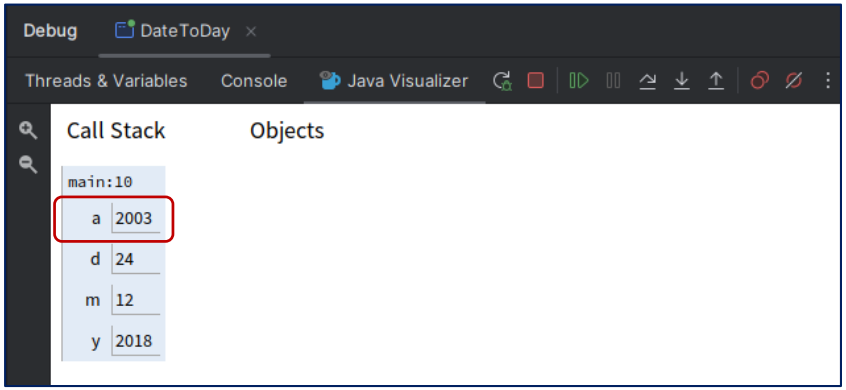


Step Into

- Click **Step Into** to execute a line of the program, in this case, line 9



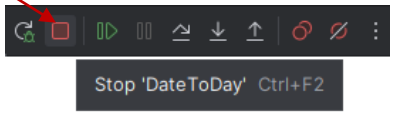
- We learn that the result of a is 2003, instead of the correct value 2018



- Stop debugging by clicking **Stop**, and fix the code

add parentheses

```
int a = y - (14 - m) / 12;
```



Debugging (again)

- After fixing the (first) bug, the final output is still incorrect: `It's day 5 !`

- Repeat the step, but set the breakpoint to line 10

```
9   int a = y - (14 - m) / 12;
10  int b = a + a/4 - a/100 + a/400;
11  int c = m + 12 * (14 - m / 12) - 2;
12  int day = (d + b + 31 * c /12) % 7;
```

- We find out that a and b are now correct, but **c is incorrect**
  - the value of is 166 (instead of the correct value 10)

- Fix the bug in line 11, and now we have the correct output

`It's day 1 !`

Call Stack

main:12

a	2018
b	2507
c	166
d	24
m	12
y	2018

Interactive User Input

- Finally, we use scanner object to get user input with the following steps:

```java
import java.util.Scanner;

public class DateToDay {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        String input = scanner.nextLine();
        int y = Integer.parseInt(input);
        input = scanner.nextLine();
        int m = Integer.parseInt(input);
        input = scanner.nextLine();
        int d = Integer.parseInt(input);
        int a = y - (14 - m) / 12;
        int b = a + a/4 - a/100 + a/400;
        int c = m + 12 * ((14 - m) / 12) - 2;
        int day = (d + b + 31 * c / 12) % 7;
        System.out.println("It's day " + day + " !");
    }
}
```

need to import Scanner library

create a Scanner object, **<u>one object</u> is enough**!

call **nextLine()** to read one line of input and to return it as a String

convert the String to the correct data type (if necessary)

repeat nextLine() and type conversion for **each** *line* of input

Input Stream

- In LM, if you use **multiple Scanner** objects, you will get an error message:

```java
import java.util.Scanner;

public class DateToDay {
    public static void main(String[] args) {
        Scanner scanner1 = new Scanner(System.in);
        String input = scanner1.nextLine();
        int y = Integer.parseInt(input);
        Scanner scanner2 = new Scanner(System.in);
        input = scanner2.nextLine();
        int m = Integer.parseInt(input);
        Scanner scanner3 = new Scanner(System.in);
        input = scanner3.nextLine();
        int d = Integer.parseInt(input);

        int a = y - (14 - m) / 12;
        int b = a + a / 4 - a / 100 + a / 400;
        int c = m + 12 * ((14 - m) / 12) - 2;
        int day = (d + b + 31 * c / 12) % 7;
        System.out.println("It's day " + day + " !");
    }
}
```

Check

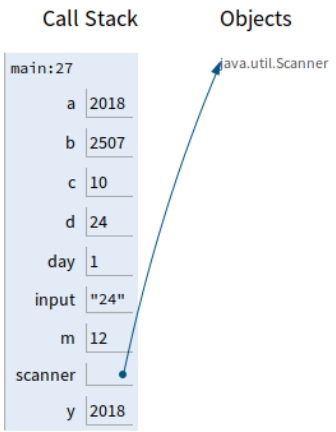
	Input	Expected	Got
✖	2018 12 24	It's day 1 !	***Run error*** Exception in thread "main" java.util.NoSuchElementExcepti at java.base/java.util.Scanner.nextLine(Scanner.j at DateToDay.main(DateToDay.java:9)

Testing was aborted due to error.

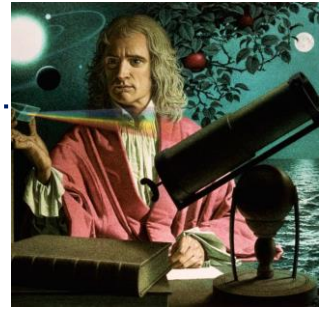
but why does this not happen in our computer?

there can be many input streams, need to just listen and read from one input stream,
just use one Scanner object!

- Visualize it!



In-Class Quiz 9: Have you learned something today?



- How to write this formula in Java?

$$F = \frac{G \times m_1 \times m_2}{r^2}$$

  - `double F = G * m1 * m2 / r^2;`
  - `double F = G * m1 * m2 / r**2;`
  - `double F = G * m1 * m2 / r * r;`
  - `double F = G * m1 * m2 / (r * r);`

Thank you for your attention !

- **Conclusion** – In this lecture, you have learned:
  - about frequently used data types and their operations
  - to use Math libraries to solve mathematical problems
  - to convert String and numbers, and vice versa
  - to debug code efficiently using Debugger
  - to observe program state using Visualizer
  - to use Scanner object to get interactive user input from keyboard

- Please continue to online **Exercises** and **Lab 2** to complete Lab Exercises and Challenge Task for CW2